

AI Agent 的最后一公里身份危机

零信任、ABAC/PBAC 与 Vault 凭证中介：精简版

赛博门外憨 & Codex 2026-06-04



视频：赛博门外憨，2026-05-23，11:23。 链接：<https://www.bilibili.com/video/BV1BxGq6PEVx/> 字幕来源：B 站自动中文字幕；英文字幕已请求，平台未产出英文 SRT。

精简版阅读目标

这版只保留一条主线：AI Agent 把用户目标、模型推理和工具调用接到企业旧系统时，最后一跳常把身份、意图、上下文和委托压成共享凭证。后端因此缺少零信任所需输入。可行治理路径是：ABAC/PBAC 重建策略字段，Vault 控制凭证置换，短期凭证缩小滥用窗口，遥测反馈推动权限收缩。

目录

1 问题：最后一公里的身份鸿沟	1
2 链路：用户目标怎样进入旧系统	2
3 坍塌：最后一跳丢掉四类语义	3
4 后果：零信任断点与攻击面放大	4
5 修复：ABAC/PBAC 与 Vault 控制桥	5
6 闭环：短期凭证与遥测反馈	7
7 总结：一条可执行的治理链	8

1 问题：最后一公里的身份鸿沟

视频的问题很集中：AI Agent 已经能理解目标、规划步骤、调用工具，企业却仍要把这些动作接入长期运行的旧系统。难点出现在最后一跳。主干能力已经很强，入户语义没有跟上。

最后一公里原本来自宽带入户类比：主干网很快，老房子的管线和入口条件却决定用户能否真正用上高速网络。映射到 AI Agent，主干网对应模型推理和工具编排，老房子对应企业旧系统、固定 API、

服务账号和历史凭证模型。

核心判断

Agent 的可用安全能力由最弱环节限制：

$$\text{可安全执行能力} \leq \min(C_{\text{reason}}, C_{\text{integration}}, C_{\text{identity}}, C_{\text{execution}})$$

其中 C_{reason} 是推理能力, $C_{\text{integration}}$ 是系统集成能力, C_{identity} 是身份与委托语义传递能力, $C_{\text{execution}}$ 是后端执行控制能力。模型越聪明, 最后一跳的身份坍塌越危险, 因为错误会直接进入真实系统。

Section 1 的问题三角：AI Agent 安全从哪里裂开

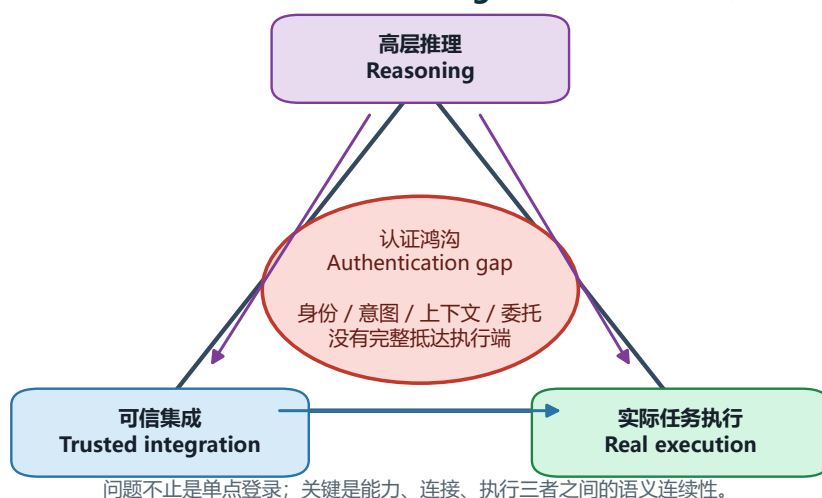


图 1: 问题三角：高层推理、可信集成与实际任务执行之间的身份鸿沟。

这个问题超出单纯登录。登录只说明某一刻用户通过了身份认证。Agent 执行链还需要回答更细的问题：用户授权 Agent 做什么、访问哪个资源、在什么上下文中访问、后端如何确认这次委托仍然有效。若这些字段没有抵达后端, 所谓“可信集成”只剩应用凭证级连通。

2 链路：用户目标怎样进入旧系统

视频中的执行路径可以压缩成一条链：

User → Chat/UI → Agent → LLM → MCP → Workflow/Data → Legacy Systems

每一段的角色如下。用户给出自然语言目标；Chat/UI 承载交互；Agent 把目标转成任务；LLM 提供理解、规划和工具选择；MCP 类服务器把 Agent 请求接到流程或数据；最终, 企业旧系统执行真实读写、审批、查询或变更。

关键边界

MCP 类服务器在这里指 Agent 与工具、数据、流程之间的中介连接层。**遗留系统**指长期运行、按应用访问模型设计、未原生承载 Agent 委托语义的企业系统。风险集中在 MCP/流程连接进入旧系统的边界, 因为后端通常习惯验证应用凭证, 缺少用户级与任务级语义。



Section 2 的核心是先把请求路径画清楚：自然语言目标进入 Agent 世界，经过推理、连接层、流程/数据，最后抵达按传统应用访问模型设计的系统。

图 2: 从用户目标到企业旧系统的 Agent 执行链。

这一链路解释了为什么“Agent 安全”不能只盯模型输出。真正有破坏力的是模型输出进入工具层之后的状态变化。只要后端动作会改密码、改数据、查敏感记录、触发审批或串联流程，系统就必须把自然语言目标转成可验证的访问字段。

3 坍塌：最后一跳丢掉四类语义

视频的机制核心是语义坍塌：链路左侧仍知道用户是谁、想做什么、处在什么环境、是否授权 Agent 代办；到了传统后端，很多请求只剩 API key 或共享凭证。

$$(s, a, r, c, d) \longrightarrow \text{API key} / \text{shared credential}$$

- s : 主体，真实用户、会话、Agent 标识或服务身份。
- a : 动作，例如读取、更新、改密码、提交审批。
- r : 资源，例如数据、业务对象、API、流程或后端系统。
- c : 上下文，例如来源、时间、设备、环境、风险状态、会话状态。
- d : 委托，用户授权 Agent 代表自己工作的范围和证据。

3.1 身份丢失

身份认证回答“用户是否登录”。身份上下文回答“这次请求能否携带该用户、会话、组织环境和 Agent 绑定关系”。后端只看服务账号或 API Key 时，审计日志会出现同一个服务主体完成大量操作，真实用户与会话被遮蔽。

3.2 意图丢失

意图是用户想完成的业务目标，动作是系统接口实际执行的操作。例如“恢复自己的账号控制权”会落成 `change-password`，“修正客户地址”会落成 `updateCustomerRecord`。后端只看动作时，无法判断动作是否仍忠于用户原始目标。

3.3 上下文丢失

上下文决定访问是否合理：来源是否异常、环境是否跨域、资源是否敏感、会话是否仍可信、风险是否升高。API key 像车钥匙，上下文像路况、限速和目的地约束。只验钥匙会错过高风险通行条件。

最后一跳把审计语义压成一个后端凭证

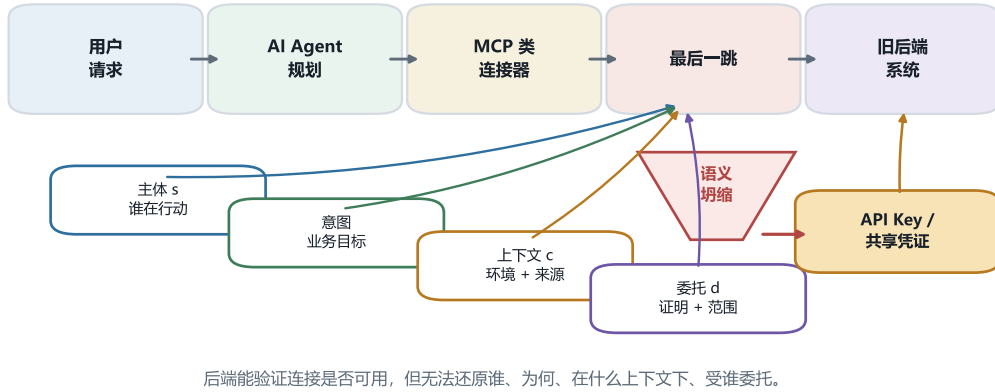


图 3: 最后一跳的语义坍塌：身份、意图、上下文和委托被压成共享凭证。

3.4 委托丢失

委托说明用户是否允许 Agent 代办、允许范围多大、何时失效。用户登录不自动授予 Agent 全域访问权；Agent 持有连接凭证也不自动拥有用户授权。没有委托证据，后端只能相信连接器本身。

压缩后的危险

API key 与共享凭证回答“连接方是否有钥匙”。Agent 执行需要回答“谁授权谁，在什么上下文中，为了哪个目标，对哪个资源做什么”。这两类问题粒度不同。粒度错配会把安全决策压回粗粒度放行。

4 后果：零信任断点与攻击面放大

零信任的最低要求是持续验证、最小权限和上下文感知访问决策。最后一跳只剩共享凭证时，后端缺少持续验证所需输入，也难以按当前任务收缩权限。

$$\{\text{身份, 意图, 上下文, 委托}\} \rightarrow \{\text{共享凭证}\} \rightarrow \text{零信任断点}$$

4.1 链式工具调用扩大横向路径

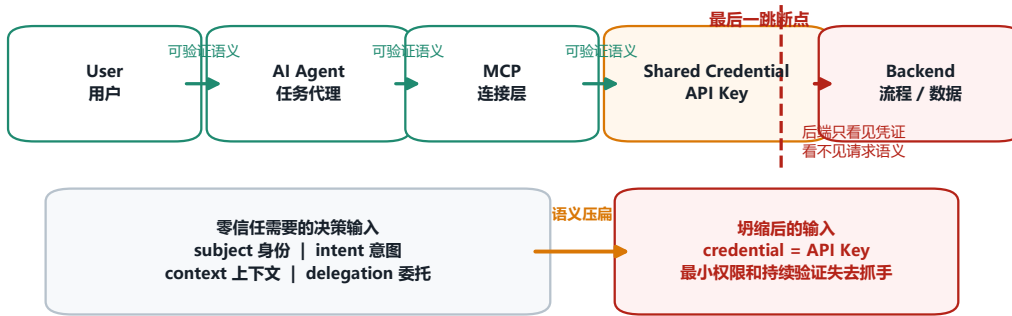
Agent 的价值在于能够连续调用工具、API 和流程。安全风险也来自同一个能力。若每个工具都以长期凭证接入，Agent 可以把账户系统、工单系统、数据平台和审批流程串成跨系统路径。

$$R \uparrow \quad \text{when} \quad (s, a, r, c, d) \downarrow \quad \text{and} \quad n_{\text{tools}} \uparrow$$

其中 R 是系统级风险， n_{tools} 是可连接工具数量。直觉很简单：字段越少，工具越多，可组合路径越多。安全审查若只看单个 API，容易漏掉 Agent 把多个 API 顺序组合后的真实攻击面。

零信任验证链在最后一跳断裂

前半段仍能携带语义；进入传统后端前，身份、意图、上下文、委托被压缩成共享凭证



教学要点：问题不在“是否登录过”，而在每一次后端访问是否还能重新验证谁、为何、在什么环境、受谁委托。

图 4：零信任验证链在传统后端边界处断裂。

4.2 Rogue Agent 的接入场景

Rogue Agent 是恶意或未授权的代理。它可以声称自己是可信代理，请求连接 MCP 与后端流程。若系统只检查连接凭证，它会错过三类验证：代理身份是否可信，用户委托是否存在，后续调用路径是否仍在策略范围内。

风险放大链

字段丢失 → 零信任断点 → 链式工具调用 → 系统级攻击面

单个共享凭证已经是风险；处在 Agent 工具链中时，它会被规划能力、工具选择和跨系统调用继续放大。

5 修复：ABAC/PBAC 与 Vault 控制桥

解决方向是把最后一跳重新变成可决策的字段集合。视频提到基于属性的访问控制和基于策略的访问控制；字幕中出现的“ABC/PBC”更合理地规范为 ABAC/PBAC。

- **ABAC**: Attribute-Based Access Control, 基于属性的访问控制。它关注主体、动作、资源、环境、风险等属性。
- **PBAC**: Policy-Based Access Control, 基于策略的访问控制。它关注规则如何集中表达、评估、审计和复用。
- **PDP**: Policy Decision Point, 策略决策点，负责计算允许、拒绝、升级验证、限制范围或签发凭证。
- **PEP**: Policy Enforcement Point, 策略执行点，负责拦截请求并落实 PDP 结果。
- **Vault**: 凭证保险库和控制中介，负责保管长期秘密、执行凭证置换、绑定策略结果与后端访问。

访问决策可以压缩成：

$$D(s, a, r, c, d, p) \rightarrow o, \quad o \in \{\text{Allow, Deny, StepUp, Limit, IssueToken}\}$$

这里 p 是策略集合， o 是决策输出。这个公式的价值在于强迫系统保留决策输入。少了主体、上下文或委托，策略就会退化“凭证存在即可访问”。

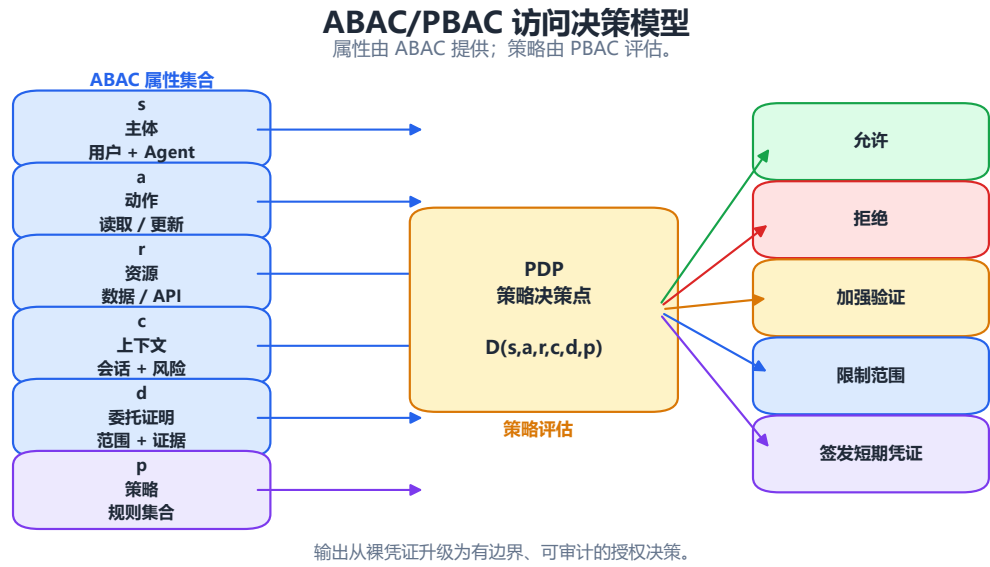


图 5: ABAC/PBAC 将请求转成可审计的访问决策。

5.1 Vault 的实际角色

Vault 需要承担三件事。第一，隔离长期凭证，让 Agent 无需直接持有数据库、工单系统或业务 API 的长期秘密。第二，承接策略结果，把身份、上下文、委托和请求范围纳入凭证发放。第三，作为后端访问桥，把 Agent 世界的语义转成旧系统可接受的受控访问。

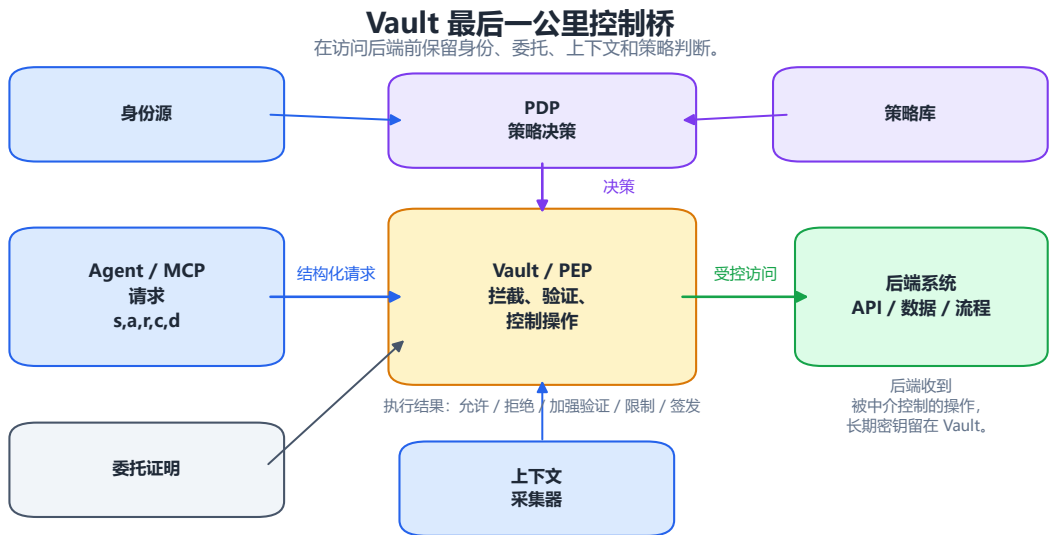


图 6: Vault/PEP 位于 Agent/MCP 与传统后端之间，控制凭证与操作。

部署前提

Vault 本身不会自动带来安全。它需要可信身份源、可传递委托证明、足够细的策略、可审计日志、后端对短期凭证或代理访问的支持。缺少这些前提，Vault 会变成新的集中风险点。

6 闭环：短期凭证与遥测反馈

长期 API key 和共享凭证的问题在于暴露窗口长、权限边界宽、用户语义弱。Vault 的治理价值来自凭证置换：系统先验证用户、意图、上下文和委托，再为当前请求换出短期凭证。

$$\text{Request}(s, a, r, c, d) \rightarrow \text{Vault/PDP} \rightarrow \tau_{\text{short}} \rightarrow \text{Backend}$$

短期凭证 τ_{short} 应满足更短生命周期、更窄作用域和更强审计绑定。可以用有效期约束表达：

$$\text{ttl}(\tau_{\text{short}}) \leq \min(T_{\text{task}}, T_{\text{risk}}, T_{\text{policy}})$$

其中 T_{task} 是完成当前任务的合理上限， T_{risk} 是当前风险水平允许的最长暴露时间， T_{policy} 是组织策略允许的最长有效期。

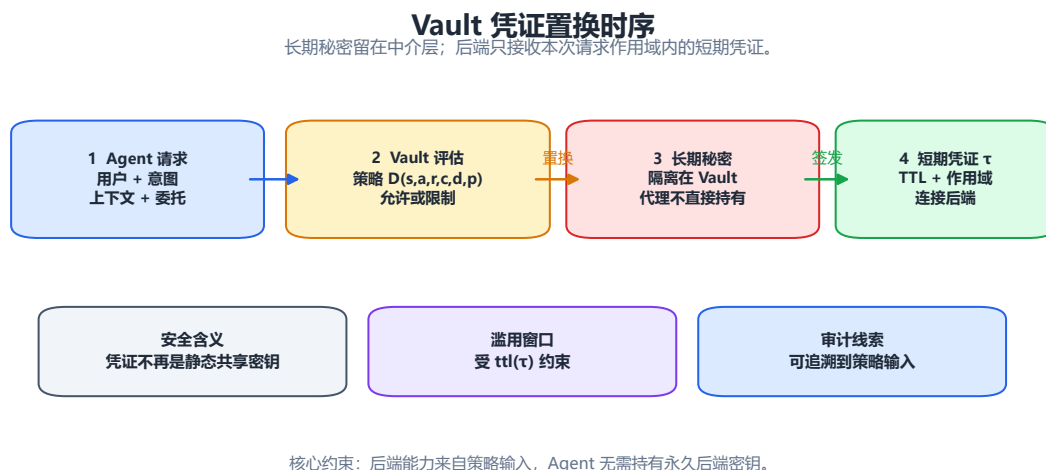


图 7: Vault 将长期凭证隔离在中介层内，向后端换出短期凭证。

凭证签发逻辑可以写成：

$$\tau_{\text{short}} = \text{IssueToken}(s, a, r, c, d, p, \text{ttl}) \quad \text{if} \quad D(s, a, r, c, d, p) = \text{IssueToken}$$

这一步把凭证从静态秘密变成策略结果。若策略输出是 Deny、StepUp 或 Limit，Vault 应阻断、要求追加验证或缩小后端访问范围。

6.1 遥测让权限可以收缩

遥测数据是运行时观察：谁发起请求，Agent 调用了哪些工具，PDP 如何判断，Vault 发放了什么凭证，后端执行了什么，后续是否出现异常路径。它不应只进入报表；它必须回到策略和 Vault。

$$p_{t+1} = \text{update}(p_t, T_t, O_t), \quad \text{Vault}_{t+1} = \text{enforce}(p_{t+1})$$

其中 T_t 是第 t 轮遥测， O_t 是对遥测的解释结果，例如正常、异常、需要撤销、需要升级验证。闭环的落点是权限收缩：拒绝、撤销、缩小作用域、缩短 TTL，或限制下一次访问。

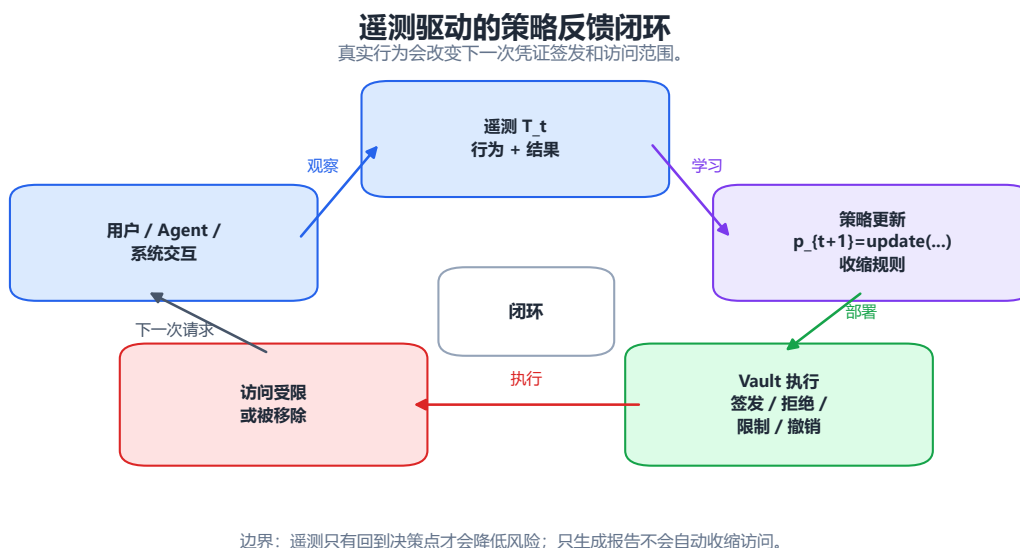


图 8: 遥测进入策略，策略回到 Vault，形成运行时权限收缩。

7 总结：一条可执行的治理链

整支视频可以浓缩为一条机制链：

语义丢失 → 决策退化 → 风险放大 → 策略桥接 → 短期凭证 → 遥测反馈 → 权限收缩



图 9: 全篇机制总图：从语义丢失到策略/Vault/遥测闭环。

最终要点

- AI Agent 的最后一公里身份危机，本质是 s, a, r, c, d 在传统凭证边界处坍塌。
- 共享 API key 能解决连通性，无法充分表达用户身份、业务意图、上下文和委托范围。
- 零信任需要持续验证、最小权限和上下文感知访问决策；这些都依赖可验证字段。

- ABAC/PBAC 提供策略输入和决策模型，Vault 提供凭证隔离、置换和执行桥。
- 短期凭证缩小滥用窗口，遥测反馈把运行行为带回策略，最终推动权限收缩。

实际落地时，团队应优先检查四个问题。第一，后端能否把请求关联到真实用户和 Agent 实例。第二，系统能否证明用户委托 Agent 执行当前动作。第三，策略是否按主体、动作、资源、上下文和委托做出细粒度判断。第四，凭证是否短期、可撤销、可审计，并能被遥测反馈持续收紧。

容易漏掉的现实约束

企业旧系统可能无法原生接受短期凭证，MCP 连接器可能缺少委托证明格式，日志可能只能记录服务账号，策略引擎可能拿不到足够上下文。这些约束会直接决定治理方案的上限。若不先盘清这些前提，ABAC、PBAC 和 Vault 会停留在架构图层面。